Manual de Laboratorio: Bootstrapping y Bare Metal en RP2040

Laboratorio de Sistemas Operativos
Facultad de Ciencias

1. Objetivos de la Práctica

1. **Comprender el Bootstrapping:** Entender qué sucede antes de que exista un **main()** o un Sistema Operativo.
2. **Dominar E/S Mapeada en Memoria:** Controlar el hardware escribiendo directamente en direcciones físicas de memoria, sin librerías de abstracción (como **gpio_init**).
3. **Cross-Compilation:** Configurar un entorno de desarrollo profesional para sistemas embebidos en Windows y Linux.

2. Requisitos y Material

- Raspberry Pi Pico (RP2040).
  - *Nota:* Si usas **Pico W**, necesitarás un LED externo obligatoriamente (el LED integrado tiene una conexión interna diferente no cubierta en este nivel básico).
- Cable Micro-USB de datos.
- Computadora (Windows 10/11 o Linux).
- LED, Resistencia (220Ω o 330Ω) y Protoboard (Opcional para Pico normal, **Obligatorio** para Pico W).

3. Configuración del Entorno (Toolchain)

El éxito de la práctica depende de una buena configuración. Sigue la sección correspondiente a tu Sistema Operativo.

4. Ejercicio: "Tu Número de Cuenta en Binario"

Para demostrar tu dominio manipulando múltiples salidas simultáneamente, realizarás la siguiente secuencia:

4.1. Para Usuarios de WINDOWS

Configuraremos el entorno manualmente descargando el SDK oficial.

Requisitos previos: Asegúrate de tener instalados en tu computadora **CMake**, el compilador **ARM GCC** y una herramienta de construcción como **Make** (MinGW) o **Ninja**.

1. **Descargar el SDK (Software Development Kit):**

- Accede al repositorio oficial: `https://github.com/raspberrypi/pico-sdk/tree/master`
- Haz clic en el botón verde **Code** y selecciona **Download ZIP** (o clónalo si usas Git).
- Descomprime el archivo en una ruta corta y sin espacios, por ejemplo: **C:\pico\pico-sdk**.

2. **Configurar la Variable de Entorno:** Para que CMake encuentre el SDK, necesitas definir la variable **PICO_SDK_PATH**. Puedes hacerlo en la terminal antes de compilar.

   Opción PowerShell:

```
$env:PICO_SDK_PATH=C:\pico\pico-sdk
```

   Opción CMD:

```
set PICO_SDK_PATH=C:\pico\pico-sdk
```

   (Nota: Ajusta la ruta a la carpeta real donde descomprimiste el archivo).

3. **El archivo CMake auxiliar:** El archivo **pico_sdk_import.cmake** necesario pueden copiarlo de la carpeta **external** dentro del SDK descargado.

4.2. Para Usuarios de LINUX (Debian/Ubuntu/Mint)

Abre una terminal y ejecuta los siguientes comandos para instalar las herramientas manualmente:

1. **Instalar dependencias:**

```
sudo apt update
sudo apt install cmake gcc-arm-none-eabi libnewlib-arm-none-eabi build-
    essential python3
```

2. **Clonar el SDK:**

```
cd ~
mkdir pico
cd pico
git clone https://github.com/raspberrypi/pico-sdk.git
cd pico-sdk
git submodule update --init
```

3. **Configurar la variable de entorno:**

```
export PICO_SDK_PATH=~/pico/pico-sdk
```

   *(Tip: Agrega esa línea al final de tu archivo ~/**.bashrc** para que sea permanente).*

5. Estructura del Proyecto

Crea una carpeta nueva llamada **BareMetal_Pico** en tus Documentos. Dentro de ella, deberás crear **tres archivos de texto** con los siguientes nombres y contenidos exactos.

5.1. Archivo 1: pico_sdk_import.cmake

Este archivo ayuda a CMake a encontrar el SDK. Descárgalo de internet (repositorio **pico-sdk/external**) o copia el archivo que viene con el SDK. No es necesario modificarlo, pero debe estar presente en la carpeta del proyecto.

5.2. Archivo 2: CMakeLists.txt (El Constructor)

Este archivo reemplaza al Makefile manual. Se encarga de ensamblar el código y, lo más importante, **añade automáticamente el Boot Stage 2** (la firma digital necesaria para que la Pico arranque).

```
cmake_minimum_required(VERSION 3.13)
include(pico_sdk_import.cmake)

project(bare_metal_blink C CXX ASM)
set(CMAKE_C_STANDARD 11)
set(CMAKE_CXX_STANDARD 17)

# Inicializar el entorno del SDK
pico_sdk_init()

# Definir el ejecutable con nuestro codigo fuente
add_executable(blink
    main.c
)

# --- PUNTO CRITICO ---
# Enlazamos 'pico_stdlib' SOLO para que el Linker encuentre funciones base
# del sistema (_exit, bootrom). NO violamos la regla de Bare Metal
# porque NO incluiremos <pico/stdlib.h> en nuestro codigo C.
target_link_libraries(blink
    pico_stdlib
)

# Generar archivos de salida (.uf2, .hex, .map)
pico_add_extra_outputs(blink)
```

5.3. Archivo 3: main.c (El Código Bare Metal)

Regla de Oro: Prohibido usar **#include <pico/stdlib.h>**. Todo el control se hará mediante punteros y direcciones de memoria del datasheet.

Corrección Importante: Manuales antiguos listan la dirección de resets como **0x40000000**. Eso es incorrecto (es SYSINFO). La dirección real es **0x4000C000**.

```
#include <stdint.h>

// === MAPA DE MEMORIA (RP2040 Datasheet) ===

// 1. SUBSYSTEM RESETS (Base: 0x4000C000)
// Controla que perifericos estan despiertos o dormidos.
#define RESETS_BASE     0x4000C000
#define RESETS_RESET    (*(volatile uint32_t *)(RESETS_BASE + 0x0))
#define RESETS_DONE     (*(volatile uint32_t *)(RESETS_BASE + 0x8))

// 2. IO BANK 0 (Base: 0x40014000)
// Controla la funcion logica de los pines (GPIO, UART, etc).
#define IO_BANK0_BASE   0x40014000
// El pin 25 tiene un offset de 0x0CC
#define GPIO25_CTRL     (*(volatile uint32_t *)(IO_BANK0_BASE + 0x0CC))

// 3. SINGLE-CYCLE IO (SIO) (Base: 0xD0000000)
// Controla el estado alto/bajo de los pines de forma rapida.
#define SIO_BASE        0xD0000000
#define SIO_GPIO_OE     (*(volatile uint32_t *)(SIO_BASE + 0x020)) // Output
    Enable
#define SIO_GPIO_OUT    (*(volatile uint32_t *)(SIO_BASE + 0x010)) // Output
    Value
```

```c

// NOTA: Si usas Pico W, cambia 25 por un pin externo (ej. 16)
#define LED_PIN         25

int main() {
    // --- PASO 1: BOOTSTRAPPING (Despertar Hardware) ---
    // Debemos quitar el Reset a dos componentes:
    // Bit 5: IO_BANK0 (Logica de pines)
    // Bit 8: PADS_BANK0 (Electricidad de pines)
    // Escribimos 0 en esos bits para sacarlos del reset.
    RESETS_RESET &= ~((1 << 5) | (1 << 8));

    // Esperamos hasta que el hardware confirme que desperto
    while ((RESETS_DONE & ((1 << 5) | (1 << 8))) != ((1 << 5) | (1 << 8)));

    // --- PASO 2: CONFIGURACION ---
    // Seleccionamos la Funcion 5 (SIO - Software Control) para el Pin 25
    GPIO25_CTRL = 5;

    // Habilitamos el Pin 25 como SALIDA en el SIO
    SIO_GPIO_OE |= (1 << LED_PIN);

    // --- PASO 3: BUCLE INFINITO (Kernel Loop) ---
    while (1) {
        // ENCENDER: Usamos XOR (^) para invertir el estado actual
        SIO_GPIO_OUT ^= (1 << LED_PIN);

        // RETARDO: Ciclo vacio. Usamos 'volatile' para que el compilador
        // no elimine este bucle.
        for (volatile int x = 0; x < 10; x++)
        {
            for (volatile int i = 0; i < 500000; i++);
        }
    }

    return 0;
}
```

6. Compilación y Ejecución

6.1. En Windows

1. Abre tu terminal (CMD o PowerShell) o la terminal de VS Code.
2. Navega a tu carpeta: **cd C:\Usuarios\TuNombre\Documentos\BareMetal_Pico**.
3. Crea el directorio de compilación:

```
mkdir build
cd build
```

4. Genera los archivos de construcción (ejemplo con MinGW):

```
cmake -G MinGW Makefiles ..
```

5. Compila:

```
mingw32-make
```

6.2. En Linux

1. Abre tu terminal en la carpeta del proyecto.
2. Crea el directorio de compilación:

```
mkdir build
cd build
```

3. Genera los Makefiles:

```
cmake ..
```

4. Compila:

```
make
```

6.3. Resultado Final

Si todo salió bien, verás un mensaje **[100 %] Built target blink**. En la carpeta **build** encontrarás el archivo **blink.uf2**.

1. Conecta tu Pico manteniendo presionado el botón **BOOTSEL**.
2. Suelta el botón cuando aparezca la unidad USB **RPI-RP2**.
3. Arrastra el archivo **blink.uf2** a esa unidad.
4. La Pico se reiniciará y el LED parpadeará.

7. Ejercicio: Tu Número de Cuenta en Binario

Para demostrar tu dominio manipulando múltiples salidas simultáneamente, realizarás la siguiente secuencia:

1. **Conexión de Hardware:**
   - Conecta 4 LEDs externos a pines consecutivos (por ejemplo: GPIO 16, 17, 18 y 19).
   - Estos representarán un número de 4 bits (Bit 0, 1, 2 y 3).
2. **Programación:**
   - El programa debe mostrar secuencialmente los dígitos de tu número de cuenta.
   - Los 4 LEDs deben encenderse o apagarse para representar el valor binario de cada dígito.
     - *Ejemplo:* Si el dígito es **5** (binario **0101**), deben estar encendidos el 1º y 3º LED, y apagados el 2º y 4º.
     - *Ejemplo:* Si el dígito es **7** (binario **0111**), deben encenderse los tres primeros LEDs.
3. **Requisitos:**
   - **Tiempo:** Debes dejar un retardo de aproximadamente **0.5 segundos** entre cada número con una pausa (apagar brevemente todos los LEDs) entre dígitos.
   - **Restricción:** No puedes usar funciones como **sleep_ms**.

8. Criterios de Evaluación

La calificación final de la práctica se compone de la siguiente manera:

- **30 % - Actividad Base:** Lograr que el LED parpadee correctamente siguiendo la guía (validación en clase).
- **30 % - Ejercicio Número de Cuenta:** Funcionamiento correcto de la secuencia binaria en los 4 LEDs.
- **40 % - Reporte y Código:**
  - El código debe estar **bien comentado**, explicando qué hace cada registro manipulado.
  - El reporte debe seguir el formato establecido en el curso (Introducción, Desarrollo, Evidencias, Conclusiones).

8.1. Modalidad de Entrega (Video)

Solo para alumnos que NO validaron la práctica presencialmente en clase.
Deberán entregar un video tipo **tutorial** (adicional al reporte y código) donde enseñen cómo realizar la práctica. No se trata solo de mostrar el hardware funcionando, sino de explicar el cómo.

Contenido del Video:

1. **Configuración del Entorno:** Explicar brevemente cómo prepararon el toolchain (SDK, CMake).
2. **Explicación del Código Base:** Describir qué hacen los registros **RESETS**, **IO_BANK0** y **SIO**.
3. **Explicación de CMake:** Qué es y por qué se usa en lugar de un Makefile manual.
4. **Demostración del Ejercicio:** Explicar la lógica implementada para el número de cuenta y mostrar los LEDs funcionando.

Rúbrica del Video:

- **Calidad Didáctica (Tutorial):** El alumno explica los conceptos con claridad y estructura lógica, no como un relato de hechos.
- **Dominio Técnico:** Uso correcto de la terminología (registros, direcciones, punteros, bits).
- **Demostración:** Se aprecia claramente el funcionamiento del hardware y la secuencia solicitada.

9. Enlaces de Interés

- **Datasheet RP2040:** `https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008371-DS-1-rp2040-datasheet.pdf`
- **Repositorio Oficial del SDK:** `https://github.com/raspberrypi/pico-sdk/tree/master`
- **Armv6-M Architecture Reference Manual:** `https://developer.arm.com/documentation/ddi0419/e/?lang=en`